Secure SCP Configurations

Preventing Over-Permissive IAM via Service Control Policies

Covers: 20 production-ready SCP policies across 6 categories
IAM hardening · Audit protection · Data protection · Network egress · Region control · Org governance

OU deployment strategy · Misconfig prevention map · Implementation checklist

SERVICE CONTROL POLICIES

Contents

SECTION 01

SCP Fundamentals & Deployment Strategy

Service Control Policies (SCPs) are the most powerful preventive control in AWS. They define the maximum permissions boundary for every principal in an AWS Organizations member account — including the root user, all IAM users, roles, and even federated identities. Unlike IAM policies, SCPs cannot be bypassed by account administrators.

> **NOTE** SCPs never grant permissions — they only restrict. An SCP deny overrides any IAM allow. An SCP allow does nothing unless the corresponding IAM policy also allows the action.

How SCPs work — the key mechanics

Property	Behaviour	Security implication
Applies to	All principals in member accounts: root, IAM users, roles, federated	Even a compromised root user cannot exceed SCP boundaries
Does NOT apply to	The management (master) account itself	Management account needs its own separate governance controls
Grant semantics	SCPs cannot grant permissions — only deny	You always need both SCP allow AND IAM allow for access
Deny-list vs allow-list	Deny-list: explicit Deny statements cap specific actions (recommended)	Allow-list: more restrictive, requires explicitly allowing everything
Inheritance	SCPs are inherited down the OU hierarchy	Root OU SCP applies to all accounts; OU SCP adds further restriction
Evaluation order	Explicit Deny in SCP wins before any IAM evaluation	First gate in the six-step IAM policy evaluation order

Deny-list strategy — recommended approach

The deny-list strategy starts with FullAWSAccess (everything allowed) and adds specific Deny statements for dangerous actions. This is safer than allow-list because it does not break existing workloads when new AWS services launch, and it is easier to audit and explain to stakeholders.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DenyDisableCloudTrail",
      "Effect": "Deny",
      "Action": [
        "cloudtrail:StopLogging",
        "cloudtrail:DeleteTrail",
        "cloudtrail:UpdateTrail"
      ],
      "Resource": "*"
```

```
    }
  ]
}
// Note: Always also attach FullAWSAccess SCP or accounts lose all access
```

OU hierarchy — where to attach SCPs

OU level	Attach what	Rationale
Root OU	Baseline deny-list: audit protection, root use, region restriction	Applies to every account in the org — universal floor
Security OU	Strict: deny most IAM writes, deny workload deployment	Security tooling accounts need tighter controls
Production OU	IAM hardening + data protection + egress controls	Prod accounts hold customer data — highest restriction
Staging/Dev OU	Moderate: IAM hardening, audit protection	Allows more experimentation but prevents catastrophic mistakes
Sandbox OU	Minimal: just audit protection + region restriction	Maximum flexibility for experimentation with a safety floor
Exceptions	Use SCP conditions (aws:PrincipalARN NotLike) to exempt break-glass roles	Pipeline automation roles may need specific exemptions

CRIT Always test SCPs in a non-production OU first. An incorrectly scoped SCP can silently break legitimate workloads — CloudTrail will show Deny events for legitimate calls.

SECTION 02

IAM Hardening SCPs

These SCPs directly prevent the over-permissive IAM configurations that enable privilege escalation. Deploy these on Production and Staging OUs as a minimum. Each prevents one or more of the 15 dangerous IAM misconfigurations.

IAM HARDENING CRITICAL

Deny root account credentials use

Prevents any use of root account credentials for day-to-day operations

What it prevents

- Root account API calls — any service, any region
- Root console login that is not account recovery
- Exploitation of root if credentials are leaked

Implementation notes

- Attach to all member account OUs — not the management account
- Does not affect automated AWS service-to-service calls that use root internally
- Break-glass: remove from specific account temporarily via emergency process

SCP JSON

```
{
  "Sid": "DenyRootAccountUse",
  "Effect": "Deny",
  "Action": "*",
  "Resource": "*",
  "Condition": {
    "StringLike": {
      "aws:PrincipalArn": "arn:aws:iam::*:root"
    }
  }
}
```

Prevents misconfig: Root account active keys | Root used for operations

IAM HARDENING CRITICAL

Deny IAM access key creation for human users

Forces all human access through IAM Identity Center (SSO) — eliminates long-lived keys

What it prevents

- Developers creating personal access keys
- Access keys that get leaked to GitHub/CI env vars
- Long-lived credentials exploited by automated scanners

Implementation notes

- Allowlist specific automation roles that legitimately need keys
- Ensure IAM Identity Center is deployed before enforcing this
- Phase: run Config rule first to identify existing keys, then enforce SCP

SCP JSON

```
{
  "Sid": "DenyAccessKeyCreation",
  "Effect": "Deny",
  "Action": "iam:CreateAccessKey",
  "Resource": "*",
  "Condition": {
    "ArnNotLike": {
      "aws:PrincipalARN": [
        "arn:aws:iam::*:role/AutomationServiceRole",
        "arn:aws:iam::*:role/PipelineDeployRole"
      ]
    }
  }
}
```

Prevents misconfig: Long-lived access keys for humans | Access key credential leak

---

IAM HARDENING CRITICAL

Deny iam:PassRole with wildcard resource

Blocks the #1 IAM privilege escalation path — PassRole with Resource:*

What it prevents

- Attaching AdminRole to EC2/Lambda/ECS via unscoped PassRole
- IMDS credential theft of high-privilege role
- Lateral movement via passed role permissions

Implementation notes

- This SCP restricts the service target — IAM policy must still scope the role ARN
- Review PassRole grants quarterly using IAM Access Analyzer
- Alert on PassRole + RunInstances from same principal within 5 min

SCP JSON

```
{
  "Sid": "DenyWildcardPassRole",
  "Effect": "Deny",
  "Action": "iam:PassRole",
  "Resource": "*",
  "Condition": {
    "ArnNotLike": {
      "iam:PassedToService": [
        "ec2.amazonaws.com",
        "lambda.amazonaws.com",
```

```
        "ecs-tasks.amazonaws.com"
      ]
    }
  }
}
// Complement with IAM policy scoping Resource to specific role ARN
```

Prevents misconfig:	iam:PassRole with Resource:*	PassRole privilege escalation chain

IAM HARDENING CRITICAL

Deny iam:CreatePolicyVersion outside governance role

Blocks single-API-call self-escalation to admin via managed policy rewrite

What it prevents

- Developer adding Action:* to their own attached managed policy
- Self-escalation with zero new resources and zero footprint
- SetDefaultPolicyVersion switching to permissive prior version

Implementation notes

- The exemption list should be very short — only dedicated IAM governance roles
- Governance roles themselves should require MFA via a separate SCP condition
- CloudTrail alert on CreatePolicyVersion by any principal not in exemption list

SCP JSON

```
{
  "Sid": "DenyCreatePolicyVersion",
  "Effect": "Deny",
  "Action": [
    "iam:CreatePolicyVersion",
    "iam:SetDefaultPolicyVersion",
    "iam:DeletePolicyVersion"
  ],
  "Resource": "*",
  "Condition": {
    "ArnNotLike": {
      "aws:PrincipalARN": [
        "arn:aws:iam::*:role/IAMGovernanceRole",
        "arn:aws:iam::*:role/PlatformEngineeringRole"
      ]
    }
  }
}
```

Prevents misconfig:	iam:CreatePolicyVersion unrestricted	SetDefaultPolicyVersion abuse

IAM HARDENING HIGH

Deny attaching admin-level policies directly

Prevents direct attachment of AdministratorAccess or over-permissive managed policies

What it prevents

- Self-attachment of AdministratorAccess in one API call
- Attachment of PowerUserAccess or IAMFullAccess to developer roles
- Backdoor policy attachment after initial compromise

Implementation notes

- Scope the denied PolicyARN list to the most sensitive managed policies
- Extend to customer-managed admin policies with a naming convention check
- Pair with CloudTrail alerts on AttachUserPolicy/AttachRolePolicy events

SCP JSON

```
{
  "Sid": "DenyAttachAdminPolicies",
  "Effect": "Deny",
  "Action": [
    "iam:AttachUserPolicy",
    "iam:AttachRolePolicy",
    "iam:AttachGroupPolicy"
  ],
  "Resource": "*",
  "Condition": {
    "ArnLike": {
      "iam:PolicyARN": [
        "arn:aws:iam::aws:policy/AdministratorAccess",
        "arn:aws:iam::aws:policy/IAMFullAccess",
        "arn:aws:iam::aws:policy/PowerUserAccess"
      ]
    },
    "ArnNotLike": {
      "aws:PrincipalARN": "arn:aws:iam::*:role/IAMGovernanceRole"
    }
  }
}
```

Prevents misconfig: iam:AttachUserPolicy on non-admin role | Direct admin policy attachment

IAM HARDENING HIGH

Deny inline policy creation on IAM identities

Forces all permissions through managed policies — eliminates hidden inline policy backdoors

What it prevents

- Inline policies hidden from standard IAM audits and Access Analyzer
- Backdoor permissions embedded in user/role that survive standard access reviews
- Inline policies deleted silently when the identity is deleted

Implementation notes

- Run Config rule iam-no-inline-policy-check before enforcing to baseline existing state
- Migrate existing inline policies to managed policies first
- Inline policies created by AWS services (e.g. Lake Formation) need exemption

SCP JSON

```
{
  "Sid": "DenyInlinePolicies",
  "Effect": "Deny",
  "Action": [
    "iam:PutUserPolicy",
    "iam:PutRolePolicy",
    "iam:PutGroupPolicy",
    "iam:DeleteUserPolicy",
    "iam:DeleteRolePolicy",
    "iam:DeleteGroupPolicy"
  ],
  "Resource": "*",
  "Condition": {
    "ArnNotLike": {
      "aws:PrincipalARN": "arn:aws:iam::*:role/IAMGovernanceRole"
    }
  }
}
```

Prevents misconfig: Inline policies on IAM identities | Hidden backdoor permission grants

IAM HARDENING HIGH

Deny IAM user creation in production accounts

Prevents backdoor IAM user creation post-compromise — forces SSO-only access

What it prevents

- Post-compromise persistence: CreateUser + AttachUserPolicy + CreateAccessKey
- Backdoor human users that survive remediation of the initial access vector
- Shadow IAM users created outside the standard provisioning process

Implementation notes

- Requires IAM Identity Center to be operational before enforcing
- Maintain a documented BreakGlass role for emergency console access
- CloudTrail alert: CreateUser event in prod account — nearly always suspicious

SCP JSON

```
{
  "Sid": "DenyIAMUserCreation",
  "Effect": "Deny",
  "Action": [
```

```
    "iam:CreateUser",
    "iam:CreateLoginProfile",
    "iam:UpdateLoginProfile"
  ],
  "Resource": "*",
  "Condition": {
    "ArnNotLike": {
      "aws:PrincipalARN": [
        "arn:aws:iam::*:role/IAMGovernanceRole",
        "arn:aws:iam::*:role/BreakGlassRole"
      ]
    }
  }
}
```

Prevents misconfig: IAM user creation as persistence mechanism | Backdoor credential creation

IAM HARDENING HIGH

Require MFA for sensitive IAM operations

Ensures privileged IAM actions always require a second factor — breaks phish-only attacks

What it prevents

- Password-only compromise granting full IAM admin access
- Stolen session token used to perform IAM escalation without MFA
- Federated users without MFA assumption performing IAM writes

Implementation notes

- Automation roles (Terraform, CI/CD) must be explicitly exempted
- Use BoolIfExists not Bool — Bool denies service-to-service calls that have no MFA concept
- Combine with IAM Identity Center MFA enforcement for all human logins

SCP JSON

```
{
  "Sid": "DenyIAMWritesWithoutMFA",
  "Effect": "Deny",
  "Action": [
    "iam:AttachRolePolicy",
    "iam:CreateRole",
    "iam:DeleteRole",
    "iam:UpdateAssumeRolePolicy",
    "iam:CreatePolicyVersion"
  ],
  "Resource": "*",
  "Condition": {
    "BoolIfExists": {
      "aws:MultiFactorAuthPresent": "false"
    },
    "ArnNotLike": {
      "aws:PrincipalARN": [
        "arn:aws:iam::*:role/PipelineDeployRole",
        "arn:aws:iam::*:role/TerraformRole"
```

```
      ]
    }
  }
}
```

Prevents misconfig:	**No MFA on privileged users**	**Phish-to-IAM-admin escalation**

SECTION 03

Audit & Detection Protection SCPs

An attacker's first action post-compromise is often to disable logging. These SCPs make your audit trail immutable — even a fully compromised account administrator cannot blind your detection capability.

AUDIT & DETECTION CRITICAL

Deny disabling CloudTrail

Makes the AWS API audit trail immutable — attackers cannot cover their tracks

What it prevents

- Attacker disabling CloudTrail to erase API call history post-compromise
- Deleting or modifying trail configuration to create blind spots
- Stopping logging in specific regions to hide lateral movement

Implementation notes

- No exemptions — not even the security team should be able to stop logging
- Pair with CloudTrail log file validation and S3 Object Lock on the log bucket
- Consider also denying cloudtrail:PutEventSelectors to prevent event type changes

SCP JSON

```
{
  "Sid": "DenyDisableCloudTrail",
  "Effect": "Deny",
  "Action": [
    "cloudtrail:StopLogging",
    "cloudtrail:DeleteTrail",
    "cloudtrail:UpdateTrail",
    "cloudtrail:PutEventSelectors",
    "cloudtrail:RemoveTags"
  ],
  "Resource": "*"
}
```

Prevents misconfig: Audit trail disabled post-compromise | Attacker covering tracks

AUDIT & DETECTION CRITICAL

Deny disabling GuardDuty and Security Hub

Preserves threat detection capability — attackers cannot blind real-time monitoring

What it prevents

- Disabling GuardDuty detector to prevent threat intel-based detection
- Removing GuardDuty member account from org-level management
- Disabling Security Hub standards to hide compliance findings

Implementation notes

- Extend to macie:DisableMacie, inspector2:Disable for full coverage
- No exemptions recommended — detection infrastructure should be immutable
- Alert on any attempt (even denied) — the attempt itself is a detection signal

SCP JSON

```
{
  "Sid": "DenyDisableDetection",
  "Effect": "Deny",
  "Action": [
    "guardduty:DeleteDetector",
    "guardduty:DisassociateFromMasterAccount",
    "guardduty:StopMonitoringMembers",
    "securityhub:DisableSecurityHub",
    "securityhub:DisassociateFromMasterAccount",
    "config:DeleteConfigurationRecorder",
    "config:StopConfigurationRecorder",
    "config:DeleteDeliveryChannel"
  ],
  "Resource": "*"
}
```

Prevents misconfig: Detection services disabled post-compromise | Security monitoring blindspot creation

AUDIT & DETECTION HIGH

Deny S3 log bucket modification

Protects CloudTrail and VPC Flow Log delivery buckets from tampering or deletion

What it prevents

- Deleting the CloudTrail S3 log delivery bucket to destroy evidence
- Modifying bucket policy to allow exfiltration of logs to attacker account
- Enabling public access on log buckets to leak audit evidence

Implementation notes

- Use a consistent bucket naming convention for all log buckets
- Combine with S3 Object Lock (GOVERNANCE or COMPLIANCE mode) on log buckets
- The log archive account should be in a separate OU with the strictest SCPs

SCP JSON

```
{
  "Sid": "ProtectLogBuckets",
  "Effect": "Deny",
  "Action": [
    "s3:DeleteBucket",
    "s3:DeleteBucketPolicy",
    "s3:PutBucketAcl",
    "s3:PutBucketPolicy",
```

```
    "s3:PutEncryptionConfiguration"
  ],
  "Resource": [
    "arn:aws:s3:::org-cloudtrail-logs-*",
    "arn:aws:s3:::org-vpc-flow-logs-*",
    "arn:aws:s3:::org-config-logs-*"
  ],
  "Condition": {
    "ArnNotLike": {
      "aws:PrincipalARN": "arn:aws:iam::*:role/LogArchiveAdmin"
    }
  }
}
```

Prevents misconfig: **Log destination tampering** **Audit trail destruction**

SECTION 04

Data Protection SCPs

Data protection SCPs prevent the two most common data exposure mechanisms in AWS: misconfigured S3 public access and unencrypted storage. They also prevent the data exfiltration pattern where an attacker copies data to an external S3 bucket under their control.

DATA PROTECTION — CRITICAL

Deny S3 public access configuration

Prevents any S3 bucket from becoming publicly accessible — blocks PII exposure at the org level

What it prevents

- Public ACL misconfiguration exposing customer PII and credentials
- Public bucket policy granting anonymous read/write access
- Disabling S3 Block Public Access account-level setting

Implementation notes

- Also deny s3:DeletePublicAccessBlock at the account level
- Pair with Config rule s3-bucket-public-read-prohibited for continuous monitoring
- Use Macie for content-level PII detection complementary to this access control

SCP JSON

```
{
  "Sid": "DenyS3PublicAccess",
  "Effect": "Deny",
  "Action": [
    "s3:PutBucketAcl",
    "s3:PutBucketPublicAccessBlock",
    "s3:PutAccountPublicAccessBlock"
  ],
  "Resource": "*",
  "Condition": {
    "StringEquals": {
      "s3:x-amz-acl": [
        "public-read",
        "public-read-write",
        "authenticated-read"
      ]
    }
  }
}
// Also add a blanket deny on PutAccountPublicAccessBlock with false values
```

Prevents misconfig: Public S3 bucket misconfiguration | PII data exposure via S3

DATA PROTECTION — HIGH

Deny S3 data exfiltration to external accounts

Prevents attackers copying data from production S3 to attacker-controlled buckets

What it prevents

- s3:CopyObject copying production data to external attacker bucket
- s3:PutBucketPolicy granting external account read access
- Data exfiltration via cross-account S3 replication rules

Implementation notes

- aws:ResourceOrgID condition restricts bucket policies to org-owned buckets only
- Complement with VPC S3 endpoint policy denying access to non-org buckets
- CloudTrail alert on s3:PutReplicationConfiguration to non-org destinations

SCP JSON

```
{
  "Sid": "DenyExternalS3Exfil",
  "Effect": "Deny",
  "Action": [
    "s3:PutBucketPolicy",
    "s3:PutReplicationConfiguration"
  ],
  "Resource": "*",
  "Condition": {
    "StringNotEquals": {
      "aws:ResourceOrgID": "${aws:PrincipalOrgID}"
    }
  }
}
// Use VPC endpoint policy for S3 to also restrict CopyObject at network level
```

Prevents misconfig: S3 data exfiltration to external bucket | Cross-account data theft

DATA PROTECTION HIGH

Deny unencrypted EBS and RDS storage

Enforces encryption at rest on all compute storage — prevents data recovery from raw volume

What it prevents

- Unencrypted EBS volumes storing application data and secrets
- RDS instances without encryption exposing database snapshots
- EC2 AMIs created from unencrypted volumes bypassing encryption

Implementation notes

- Enable EBS encryption by default at the account level as a complementary control
- Use KMS CMKs for encryption to maintain key control and audit trail
- Audit existing unencrypted volumes with Config rule encrypted-volumes first

SCP JSON

```
{
  "Sid": "DenyUnencryptedStorage",
  "Effect": "Deny",
  "Action": [
    "ec2:RunInstances",
    "ec2:CreateVolume"
  ],
  "Resource": [
    "arn:aws:ec2:*:*:volume/*",
    "arn:aws:ec2:*:*:instance/*"
  ],
  "Condition": {
    "Bool": {
      "ec2:Encrypted": "false"
    }
  }
}
// Also add separate statement for RDS:
// Action: rds:CreateDBInstance, Condition: rds:StorageEncrypted = false
```

Prevents misconfig:	**Unencrypted data at rest**	**Storage snapshot data theft**

SECTION 05

Network & Egress SCPs

Network SCPs limit the blast radius of a compromised workload by restricting where data can flow. They also protect VPC infrastructure from being modified to create attacker-controlled backdoors.

NETWORK & EGRESS HIGH

Deny VPC peering and internet gateway creation by workloads

Prevents workload accounts from creating uncontrolled network paths

What it prevents

- Attacker creating VPC peering to their own account for persistent access
- Rogue internet gateway bypassing network security controls
- Direct Connect gateway attachment creating exfiltration path

Implementation notes

- Network infrastructure should be provisioned only by IaC (Terraform/CDK) roles
- Use AWS Transit Gateway in a networking hub account — workload accounts should not create their own gateways
- CloudTrail alert on CreateInternetGateway by non-network roles

SCP JSON

```
{
  "Sid": "DenyRogueNetworking",
  "Effect": "Deny",
  "Action": [
    "ec2:CreateInternetGateway",
    "ec2:AttachInternetGateway",
    "ec2:CreateVpcPeeringConnection",
    "ec2:AcceptVpcPeeringConnection",
    "ec2:CreateTransitGatewayPeeringAttachment",
    "directconnect:CreateConnection"
  ],
  "Resource": "*",
  "Condition": {
    "ArnNotLike": {
      "aws:PrincipalARN": [
        "arn:aws:iam::*:role/NetworkAdminRole",
        "arn:aws:iam::*:role/TerraformNetworkRole"
      ]
    }
  }
}
```

Prevents misconfig: Rogue network path creation | Data exfiltration via new network routes

NETWORK & EGRESS CRITICAL

Deny IMDSv1 on EC2 instance launch

Forces IMDSv2 with hop-limit=1 — breaks SSRF-to-IMDS credential theft attack chain

What it prevents

- SSRF vulnerability on EC2 web app fetching 169.254.169.254 via GET only
- IMDSv1 credential theft giving attacker the EC2 role STS credentials
- Lateral movement via stolen instance role credentials used externally

Implementation notes

- Apply to all OUs — no legitimate use case for IMDSv1 in new instances
- Existing instances: use AWS Config rule ec2-imdsv2-check + remediation
- For ECS: separate control via task metadata endpoint (169.254.170.2) — use task role least privilege

SCP JSON

```
{
  "Sid": "RequireIMDSv2",
  "Effect": "Deny",
  "Action": "ec2:RunInstances",
  "Resource": "arn:aws:ec2:*:*:instance/*",
  "Condition": {
    "StringNotEquals": {
      "ec2:MetadataHttpTokens": "required"
    }
  }
}
// Also enforce hop-limit:
// "NumericGreaterThan": {"ec2:MetadataHttpPutResponseHopLimit": "1"}
```

Prevents misconfig:	**SSRF to IMDS credential theft**	**EC2 role credential exfiltration**

SECTION 06

Region & Compute SCPs

Region and compute SCPs enforce data residency requirements and limit the attacker's lateral movement surface by restricting which services can be used in which regions.

REGION & COMPUTE HIGH

Deny actions outside approved regions

Enforces data residency, limits lateral movement surface, and reduces attack exposure

What it prevents

- Attacker spinning up resources in unsupported regions to evade monitoring
- Data residency violation: customer data processed outside approved geography
- GuardDuty/Config not enabled in non-approved regions — blind spot creation

Implementation notes

- Use NotAction for global services (IAM, Route53, CloudFront) that have no region
- Approved region list should match where GuardDuty and Config are enabled
- Review and update the region list when expanding to new geographies

SCP JSON

```
{
  "Sid": "DenyOutsideApprovedRegions",
  "Effect": "Deny",
  "NotAction": [
    "iam:*",
    "organizations:*",
    "support:*",
    "sts:*",
    "cloudfront:*",
    "route53:*",
    "globalaccelerator:*",
    "waf:*"
  ],
  "Resource": "*",
  "Condition": {
    "StringNotEquals": {
      "aws:RequestedRegion": [
        "ap-south-1",
        "ap-southeast-1",
        "eu-west-1",
        "us-east-1"
      ]
    }
  }
}
```

Prevents misconfig: Resources created in unmonitored regions | Data residency violation

REGION & COMPUTE MEDIUM

Deny launch of instances without approved AMI

Restricts EC2 instance launches to approved, hardened AMIs from your account or AWS

What it prevents

- Launch of unpatched public AMIs with known CVEs as lateral movement vector
- Public AMI with attacker-embedded backdoor code running as trusted instance
- Unapproved AMIs bypassing your OS hardening and agent deployment pipeline

Implementation notes

- Maintain a golden AMI pipeline with EC2 Image Builder + Inspector scanning
- Update the approved AMI list as new hardened images are published
- This SCP is best for mature organisations — simpler orgs should start with IMDSv2 SCP first

SCP JSON

```
{
  "Sid": "DenyUnapprovedAMI",
  "Effect": "Deny",
  "Action": "ec2:RunInstances",
  "Resource": "arn:aws:ec2:*:*:instance/*",
  "Condition": {
    "StringNotLike": {
      "ec2:ImageID": [
        "ami-GOLDEN_AMI_ID_1",
        "ami-GOLDEN_AMI_ID_2"
      ]
    },
    "ArnNotLike": {
      "aws:PrincipalARN": [
        "arn:aws:iam::*:role/AMIBuildRole",
        "arn:aws:iam::*:role/EC2ImageBuilderRole"
      ]
    }
  }
}
```

Prevents misconfig: Launch of backdoored public AMI | Unpatched instance as lateral movement platform

SECTION 07

Org Governance SCPs

Governance SCPs protect the AWS Organizations structure itself. A compromised management account or a rogue account attempting to leave the organization are high-severity scenarios that these SCPs address.

ORG GOVERNANCE CRITICAL

Deny leaving the AWS Organisation

Prevents a compromised account from detaching itself to escape SCP governance

What it prevents

- Account leaving org to escape all SCP guardrails and audit controls
- Attacker with management-level access removing accounts from governance
- Rogue insider removing account to create an ungoverned shadow environment

Implementation notes

- Apply at Root OU level — single SCP protects entire organization
- No exemptions — legitimate account retirement goes through a formal process
- Pair with CloudTrail alerts on LeaveOrganization attempts

SCP JSON

```
{
  "Sid": "DenyLeaveOrganization",
  "Effect": "Deny",
  "Action": [
    "organizations:LeaveOrganization"
  ],
  "Resource": "*"
}
// Apply at Root OU — protects all member accounts simultaneously
```

Prevents misconfig: Account escaping org governance | SCP bypass via org detachment

ORG GOVERNANCE HIGH

Deny modification of AWS Config rules and conformance packs

Protects compliance monitoring infrastructure from being disabled or modified

What it prevents

- Attacker deleting Config rules to remove compliance alerting
- Modifying Config recorder to stop tracking resource changes
- Deleting conformance packs to erase compliance posture evidence

Implementation notes

- Governance roles should be in a separate Security OU with the strictest SCPs
- Config rule changes should go through IaC (Terraform/CDK) with PR review
- CloudTrail alert on any DeleteConfigRule or PutConfigRule by non-governance principal

SCP JSON

```
{
  "Sid": "ProtectConfigService",
  "Effect": "Deny",
  "Action": [
    "config:DeleteConfigRule",
    "config:DeleteConformancePack",
    "config:DeleteOrganizationConfigRule",
    "config:PutConfigRule",
    "config:PutConformancePack",
    "config:DeleteRemediationConfiguration"
  ],
  "Resource": "*",
  "Condition": {
    "ArnNotLike": {
      "aws:PrincipalARN": [
        "arn:aws:iam::*:role/SecurityGovernanceRole",
        "arn:aws:iam::*:role/TerraformSecurityRole"
      ]
    }
  }
}
```

Prevents misconfig:	Compliance monitoring disabled	Config rule tampering to hide violations

SECTION 08

OU-Level Deployment Matrix

Not all SCPs apply equally to all account types. Use this matrix to decide which SCPs to deploy at each OU level. Start with the Root OU baseline and add restrictions as you move to more sensitive environments.

SCP Policy	Root OU	Security OU	Prod OU	Dev/Stg OU	Sandbox OU
Deny root account use	Y	Y	Y	Y	Y
Deny access key creation	N	Y	Y	P	N
Deny wildcard PassRole	Y	Y	Y	Y	P
Deny CreatePolicyVersion	Y	Y	Y	Y	N
Deny AttachUserPolicy (admin)	N	Y	Y	P	N
Deny inline policies	N	Y	Y	P	N
Deny IAM user creation	N	Y	Y	N	N
Require MFA for IAM writes	N	Y	Y	Y	N
Deny disable CloudTrail	Y	Y	Y	Y	Y
Deny disable GuardDuty/SecHub	Y	Y	Y	Y	P
Protect log buckets	Y	Y	Y	Y	N
Deny S3 public access	Y	Y	Y	Y	P
Deny S3 exfiltration	N	Y	Y	P	N
Deny unencrypted storage	N	Y	Y	P	N
Deny rogue networking	N	Y	Y	P	N
Require IMDSv2	Y	Y	Y	Y	P
Deny outside approved regions	Y	Y	Y	P	N
Deny unapproved AMI	N	Y	P	N	N
Deny leave organisation	Y	Y	Y	Y	Y
Protect Config service	N	Y	Y	P	N

Y	Deploy — recommended for this OU level
P	Partial — deploy with exemptions for automation roles
N	Skip — too restrictive or not applicable for this OU

SECTION 09

Misconfig Prevention Map

This map shows which SCP directly prevents each of the 15 dangerous IAM misconfigurations. Use it to confirm your SCP baseline closes the gaps identified in an IAM security review.

#	IAM Misconfiguration	Severity	SCP that prevents it	Additional control
01	Action:* Resource:* wildcard policy	CRITICAL	Deny AttachUserPolicy (admin policies)	IAM Access Analyzer continuous monitoring
02	Root account active access keys	CRITICAL	Deny root account use	Delete keys + hardware MFA on root
03	No MFA on privileged users	CRITICAL	Require MFA for IAM writes	IAM Identity Center MFA enforcement
04	Long-lived access keys for humans	CRITICAL	Deny access key creation	SSO migration + Config rule: access-keys-rotated
05	iam:PassRole with Resource:*	CRITICAL	Deny wildcard PassRole	IAM policy: scope Resource to specific ARN
06	iam:CreatePolicyVersion unrestricted	CRITICAL	Deny CreatePolicyVersion	CloudTrail alert on CreatePolicyVersion events
07	iam:AttachUserPolicy on non-admin	HIGH	Deny AttachUserPolicy (admin)	Permission boundary on developer roles
08	Cross-account trust no ExternalId	HIGH	No direct SCP — IAM policy control	IAM Access Analyzer + trust policy review
09	OIDC role no branch condition	CRITICAL	No direct SCP — trust policy control	CloudTrail: AssumeRoleWithWebIdentity review
10	Lambda secrets in env vars	HIGH	No direct SCP — application control	Secrets Manager + deny GetFunctionConfiguration
11	iam:* on developer role	HIGH	Deny CreatePolicyVersion + Attach*	IAM Access Analyzer: unused permissions
12	Resource policy Principal:*	HIGH	Deny S3 public access (for S3)	IAM Access Analyzer: external access findings
13	Inline policies on identities	HIGH	Deny inline policies (PutUserPolicy)	Config rule: iam-no-inline-policy-check
14	No permission boundaries	MEDIUM	No direct SCP — deployment control	IaC enforcement: boundaries in Terraform modules
15	No SCP guardrails	MEDIUM	This guide IS the fix	Deploy this SCP baseline to all OUs

SECTION 10

Implementation Checklist

Use this checklist to deploy and validate your SCP baseline. Follow phases in order.

Phase 1 — Baseline safety (do first) CRITICAL

- Attach FullAWSAccess SCP to all OUs before adding any Deny SCPs [ ]
- Enable CloudTrail in all regions in all accounts [ ]
- Enable GuardDuty and Security Hub in all accounts [ ]
- Run IAM credential report and identify existing violations [ ]
- Test all SCPs in a dedicated Sandbox OU before production rollout [ ]

Phase 2 — Audit protection (week 1) HIGH

- Deploy: Deny disable CloudTrail to Root OU [ ]
- Deploy: Deny disable GuardDuty/Security Hub to Root OU [ ]
- Deploy: Deny leave organisation to Root OU [ ]
- Deploy: Protect log buckets to all OUs with log delivery [ ]
- Validate: CloudTrail shows Deny on test StopLogging call [ ]

Phase 3 — IAM hardening (week 2-3) HIGH

- Deploy: Deny root account use to all member OUs [ ]
- Deploy: Deny wildcard PassRole to Production and Staging OUs [ ]
- Deploy: Deny CreatePolicyVersion to Production and Staging OUs [ ]
- Deploy: Require IMDSv2 to all OUs [ ]
- Deploy: Deny S3 public access to all OUs [ ]
- Validate: Test each SCP with IAM policy simulator or AWS CLI call [ ]

Phase 4 — Full hardening (week 4+) MEDIUM

- Deploy remaining IAM hardening SCPs with appropriate OU targeting [ ]
- Configure exemption lists (ArnNotLike) for automation roles [ ]
- Deploy region restriction SCP after confirming workloads in approved regions [ ]
- Enable Config rules to continuously monitor SCP coverage gaps [ ]
- Schedule quarterly SCP review: update exemption lists and exemptions [ ]

Ongoing — validation and monitoring MEDIUM

- Monitor CloudTrail for Deny events from SCPs — legitimate denies indicate misconfiguration [ ]
- Review IAM Access Analyzer findings monthly [ ]
- Update approved region list when expanding to new geographies [ ]
- Review exemption lists quarterly — remove stale role ARNs [ ]
- Run Prowler quarterly against CIS AWS Benchmark [ ]

SCPs are preventive controls — they stop the mistake before it happens. Pair them with detective controls (GuardDuty, Config, CloudTrail alerts) and corrective controls (auto-remediation Lambdas) for defence in depth.